

```
"""
```

```
StegSim CLI - python -m stegsim [command] [options]
```

```
Commands:
```

```
run      - run a simulation from a config file
demo     - run the built-in stale authority drift demo
verify   - verify determinism of a config
replay   - replay receipts from a run directory
sweep    - run parameter sweep (v2: phase diagram)
```

```
"""
```

```
import argparse
```

```
import json
```

```
import sys
```

```
from pathlib import Path
```

```
def cmd_run(args):
```

```
    from .config import SimConfig
```

```
    from .run import run_simulation
```

```
    if args.config:
```

```
        cfg = SimConfig.from_file(args.config)
```

```
    else:
```

```
        cfg = SimConfig()
```

```
    if args.run_id:
```

```
        cfg.run_id = args.run_id
```

```
    if args.seed is not None:
```

```
        cfg.seed = args.seed
```

```
    if args.agents is not None:
```

```
        cfg.agents = args.agents
```

```
    if args.steps is not None:
```

```
        cfg.steps = args.steps
```

```
    if args.output:
```

```
        cfg.output_dir = args.output
```

```
    summary = run_simulation(cfg)
```

```
    print(json.dumps({
```

```
        "run_id": summary["run_id"],
```

```
        "final_viability": summary["final_viability"],
```

```
        "status": summary["final_viability_status"],
```

```
        "peak_lg_disagree": summary["peak_local_global_disagreement_rate"],
```

```
        "total_receipts": summary["total_receipts"],
```

```
    }, indent=2))
```

```

return 0

def cmd_demo(args):
    from .config import SimConfig
    from .run import run_simulation

    print("[StegSim] Running built-in demo: Stale Authority Under Environmental D
    cfg = SimConfig()
    cfg.scenario = "stale_authority_drift"
    cfg.run_id = "demo-stale-authority-001"
    cfg.seed = 1729
    cfg.agents = 1000 # small for quick demo; use 10000 for full run
    cfg.steps = 250
    cfg.output_dir = args.output or "runs"

    summary = run_simulation(cfg)

    print(f"\n{'='*60}")
    print(f"DEMO COMPLETE")
    print(f"{'='*60}")
    print(f"Scenario: {summary['scenario']}")
    print(f"Initial V: {summary['initial_viability']:.4f}")
    print(f"Final V: {summary['final_viability']:.4f} ({summary['final_via
    print(f"Min V: {summary['min_viability']:.4f} at step {summary['min_
    print(f"First warning: step {summary['first_warning_step']}")
    print(f"First inadmiss: step {summary['first_inadmissible_step']}")
    print(f"Peak LG disagr: {summary['peak_local_global_disagreement_rate']:.1%}"
    print(f"Total receipts: {summary['total_receipts']:,}")
    print(f"Report: runs/{cfg.run_id}/REPORT.md")
    print(f"{'='*60}")
    print(f"\nKey finding:")
    print(f"Local/global disagreement peaked at "
          f"{summary['peak_local_global_disagreement_rate']:.1%} -")
    print(f"meaning that fraction of proposals passed local policy")
    print(f"while failing global admissibility.")
    print(f"\n> All dashboards green. System already lost.")
    return 0

def cmd_verify(args):
    from .config import SimConfig
    from .run import verify_determinism

    cfg = SimConfig.from_file(args.config) if args.config else SimConfig()
    cfg.agents = min(cfg.agents, 500) # small for verify speed
    cfg.steps = min(cfg.steps, 50)

```

```

ok = verify_determinism(cfg, n_runs=2)
return 0 if ok else 1

def cmd_replay(args):
    """Replay receipts from a run directory and verify chain."""
    from pathlib import Path
    import json

    run_dir = Path(args.run_dir)
    receipts_path = run_dir / "receipts.jsonl"

    if not receipts_path.exists():
        print(f"No receipts.jsonl in {run_dir}")
        return 1

    receipts = []
    with open(receipts_path) as f:
        for line in f:
            line = line.strip()
            if line:
                receipts.append(json.loads(line))

    # Verify decision counts match summary
    summary_path = run_dir / "summary.json"
    if summary_path.exists():
        with open(summary_path) as f:
            summary = json.load(f)
        expected_total = summary.get("total_receipts", 0)
        if len(receipts) != expected_total:
            print(f"MISMATCH: {len(receipts)} receipts vs {expected_total} expect")
            return 1

    allow = sum(1 for r in receipts if r.get("decision") == "ALLOW")
    deny = sum(1 for r in receipts if r.get("decision") == "DENY")
    flag = sum(1 for r in receipts if r.get("decision") == "FLAG")
    fc = sum(1 for r in receipts if r.get("decision") == "FAIL_CLOSED")
    lg = sum(1 for r in receipts if r.get("local_global_disagree"))

    print(f"Replay: {len(receipts):,} receipts")
    print(f"  ALLOW:      {allow:,}")
    print(f"  DENY:      {deny:,}")
    print(f"  FLAG:      {flag:,}")
    print(f"  FAIL_CLOSED: {fc:,}")
    print(f"  LG disagree: {lg:,} ({lg/len(receipts):.2%}")
    print(f"Replay: OK")

```

```

return 0

def cmd_sweep(args):
    """
    v2: Phase diagram sweep across trust_decay * authority_staleness parameter sp
    Produces phase_diagram.jsonl for visualization.
    """
    from .config import SimConfig
    from .run import run_simulation
    from .receipts import PhaseDiagramCollector
    import itertools

    trust_decays = [0.0005, 0.001, 0.002, 0.005, 0.01]
    auth_stale = [0.001, 0.002, 0.005, 0.010, 0.02]

    output_dir = args.output or "runs/sweep"
    collector = PhaseDiagramCollector(f"{output_dir}/phase_diagram.jsonl")

    total = len(trust_decays) * len(auth_stale)
    print(f"[StegSim] Phase diagram sweep: {total} configurations")

    for i, (td, ast) in enumerate(itertools.product(trust_decays, auth_stale)):
        cfg = SimConfig()
        cfg.scenario = "phase_sweep"
        cfg.run_id = f"sweep-td{td:.4f}-ast{ast:.4f}"
        cfg.seed = 1729
        cfg.agents = 500
        cfg.steps = 200
        cfg.output_dir = output_dir
        cfg.drift.trust_decay_per_step = td
        cfg.drift.authority_staleness_per_step = ast

        print(f"[{i+1}/{total}] trust_decay={td} auth_stale={ast}")
        summary = run_simulation(cfg)

        collector.record(
            trust_decay=td,
            auth_staleness=ast,
            first_warning=summary.get("first_warning_step") or cfg.steps,
            first_inadmissible=summary.get("first_inadmissible_step") or cfg.steps,
            final_viability=summary["final_viability"],
        )

    n = collector.write()
    print(f"[StegSim] Phase diagram: {n} points written to {output_dir}/phase_dia")
    return 0

```

```

def main():
    parser = argparse.ArgumentParser(
        prog="stegsim",
        description="StegSim - Viability Boundary Drift Simulator"
    )
    subparsers = parser.add_subparsers(dest="command")

    # run
    p_run = subparsers.add_parser("run", help="Run simulation from config file")
    p_run.add_argument("--config", help="Path to config JSON file")
    p_run.add_argument("--run-id", dest="run_id", help="Override run ID")
    p_run.add_argument("--seed", type=int, help="Override random seed")
    p_run.add_argument("--agents", type=int, help="Override agent count")
    p_run.add_argument("--steps", type=int, help="Override step count")
    p_run.add_argument("--output", help="Output directory")

    # demo
    p_demo = subparsers.add_parser("demo", help="Run built-in stale authority dri
    p_demo.add_argument("--output", help="Output directory")

    # verify
    p_verify = subparsers.add_parser("verify", help="Verify determinism")
    p_verify.add_argument("--config", help="Path to config JSON file")

    # replay
    p_replay = subparsers.add_parser("replay", help="Replay receipts from run dir
    p_replay.add_argument("run_dir", help="Path to run directory")

    # sweep
    p_sweep = subparsers.add_parser("sweep", help="Phase diagram parameter sweep
    p_sweep.add_argument("--output", help="Output directory")

    args = parser.parse_args()

    if args.command == "run":
        sys.exit(cmd_run(args))
    elif args.command == "demo":
        sys.exit(cmd_demo(args))
    elif args.command == "verify":
        sys.exit(cmd_verify(args))
    elif args.command == "replay":
        sys.exit(cmd_replay(args))
    elif args.command == "sweep":
        sys.exit(cmd_sweep(args))
    else:

```

```
    parser.print_help()
    sys.exit(1)
```

```
if __name__ == "__main__":
    main()
```